॥ वसुधैव कुटुम्बकम् ॥

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re-accredited by NAAC with "A" Grade

Name of the Student: Postiwala Vatsal Dipakbhai **PRN: 26070122509**

Semester: 3 **Year: 2026-2027**

Subject Title: Operating Systems Lab	
EXPERIMENT No: 9	
TITLE: SJF Scheduling Algorithm	**DoP**: 21-09-2026

Aim: Implement a C program to demonstrate the SJF algorithm.

Learning Outcome: 1. To understand the scheduling algorithm
2. To Demonstrate the SJF algorithm

Hardware/Software:

- Hardware requirements:
  - A PC with a minimum of 4GB RAM.
  - A processor with 1.6GHz clock speed or faster.
  - Sufficient hard disk space to store the code and compiler.
- Software Requirements:
  - Visual Studio Code (VS Code)
  - C compiler such as GCC
  - C/C++ extension for VS Code. This extension is used to compile and run C++ code directly from VS Code.

Problem Statement:

Calculate following

- Gantt Chart
- Completion Time (CT)
- Turnaround Time (TAT)
- Waiting Time (WT)
- Response Time (RT)
- Average Waiting Time
- Average Turnaround Time
- Number of switches

1. **Solve using non-preemptive SJF**

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	9
P2	1	5
P3	2	2
P4	3	6
P5	5	3
P6	6	2
P7	7	4
P8	9	1
P9	11	3

2. **Solve using non-preemptive SJF**

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	7
P2	2	4
P3	4	1
P4	5	3
P5	6	2
P6	8	5
P7	10	1

Theory:

Scheduling algorithms are used in operating systems to allocate system resources to processes in an efficient manner. Scheduling algorithms determine which process gets access to the CPU and when, based on predefined criteria such as priority, execution time, and resource requirements.

Shortest Job First (SJF) is a scheduling algorithm that aims to improve system throughput and reduce the average waiting time of processes by selecting the process with the shortest execution time for execution next. There are two variants of SJF: non-preemptive SJF and preemptive SJF.

Non-preemptive SJF schedules the process with the shortest estimated burst time first, but once a process starts executing, it is not preempted. This means that the process keeps using the CPU until it completes or voluntarily gives up the CPU. Non-preemptive SJF can lead to improved system throughput and reduced waiting times for processes with large execution times compared to FCFS, but it may not be suitable for real-time systems where priority is critical.

Preemptive SJF is a dynamic priority scheduling algorithm that allows the currently executing process to be preempted if a new process with a shorter estimated burst time arrives. This means that a process can be interrupted and put on hold to execute another process with a shorter estimated burst time. Preemptive SJF can lead to further improvements in system throughput and reduced waiting times compared to non-preemptive SJF, but it requires more overhead to manage the dynamic priority of processes.

Both non-preemptive and preemptive SJF require accurate estimates of the burst time of processes to achieve efficient scheduling. If the estimated burst time is incorrect, it can lead to inefficient scheduling and increased waiting times for some processes. Moreover, SJF is not suitable for real-time systems where priority is critical and cannot be compromised for execution time.

The features of the SJF algorithm include:

- Process scheduling based on the shortest burst time
- Non-preemptive or preemptive: SJF can be implemented as a non-preemptive or preemptive scheduling algorithm
- Efficient utilization of CPU
- Simple to understand and implement
- Suitable for batch-processing systems

The drawbacks of the SJF algorithm include:

- Difficulty in estimating burst time
- Possibility of starvation: If shorter processes continually arrive, longer processes may never get a chance to execute
- Not suitable for real-time systems: SJF may not be suitable for real-time systems that require strict guarantees on response time and priority
- May suffer from convoy effect: SJF can suffer from the convoy effect, where a long process occupies the CPU and causes shorter processes to wait for an extended period, leading to increased waiting times and reduced system throughput.
- Preemptive variant may cause overhead: Preemptive SJF requires more overhead to manage the dynamic priority of processes and may cause unnecessary context switching, which can reduce system throughput.

Algorithm:

- Start.
- Read the number of processes.
- Enter the Arrival Time (AT) and Burst Time (BT) of each process.
- Set the current time to 0.
- Find all processes whose arrival time is less than or equal to the current time and which are not completed.
- Among them, select the process having the shortest Burst Time.
- If no process has arrived, increase the current time until a process arrives.
- Execute the selected process completely because SJF is non-preemptive.
- Calculate:
    - CT = Completion Time
    - TAT = CT - AT
    - WT = TAT - BT
    - RT = Start Time - AT
- Mark the process as completed.
- Repeat steps 5–10 until all processes are completed.
- Calculate average WT and average TAT.
- Count the number of process switches.
- Display the results and Gantt Chart.
- Stop.

Formulas

Completion Time

$$CT = \text{time at which process finishes}$$

Turnaround Time

$$TAT = CT - AT$$

Waiting Time

$$WT = TAT - BT$$

Response Time

$$RT = Start\ Time - AT$$

Average Waiting Time

$$Average\ WT = \frac{\sum WT}{n}$$

Average Turnaround Time

$$Average\ TAT = \frac{\sum TAT}{n}$$

Program:

```
#include <stdio.h>

void sjf(int n, int pid[], int at[], int bt[]) {
    int st[10], ct[10], tat[10], wt[10], rt[10];
    int done[10] = {0};
    int time = 0, completed = 0;
    int i, min;
    float avgWT = 0, avgTAT = 0;

    printf("Gantt Chart:\n");

    while (completed < n) {
        min = -1;

        for (i = 0; i < n; i++) {
            if (!done[i] && at[i] <= time) {
                if (min == -1 || bt[i] < bt[min])
                    min = i;
            }
        }

        if (min == -1) {
            time++;
            continue;
        }

        st[min] = time;
```

```
        ct[min] = time + bt[min];
        tat[min] = ct[min] - at[min];
        wt[min] = tat[min] - bt[min];
        rt[min] = st[min] - at[min];

        time = ct[min];
        done[min] = 1;
        completed++;

        avgWT += wt[min];
        avgTAT += tat[min];

        printf("| P%d ", pid[min]);
    }

    printf("|\n\n");

    printf("Process\tAT\tBT\tCT\tTAT\tWT\tRT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i], at[i], bt[i], ct[i],
               tat[i], wt[i], rt[i]);
    }

    printf("\nAverage WT  = %.2f", avgWT / n);
    printf("\nAverage TAT = %.2f", avgTAT / n);
    printf("\nNumber of switches = %d\n\n", n - 1);
}

int main() {

    // Question 1
    int pid1[] = {1,2,3,4,5,6,7,8,9};
    int at1[]  = {0,1,2,3,5,6,7,9,11};
    int bt1[]  = {9,5,2,6,3,2,4,1,3};

    printf("========== QUESTION 1 ==========\n\n");
    sjf(9, pid1, at1, bt1);


    // Question 2
    int pid2[] = {1,2,3,4,5,6,7};
    int at2[]  = {0,2,4,5,6,8,10};
    int bt2[]  = {7,4,1,3,2,5,1};

    printf("========== QUESTION 2 ==========\n\n");
    sjf(7, pid2, at2, bt2);

    return 0;
}
```

Output:

```
vatsal@Ubuntu-VirtualBox:~/OS$ gcc SJF.c -o SJF
vatsal@Ubuntu-VirtualBox:~/OS$ ./SJF
========== QUESTION 1 ==========

Gantt Chart:
| P1 | P8 | P3 | P6 | P5 | P9 | P7 | P2 | P4 |

Process AT      BT      CT      TAT     WT      RT
P1      0       9       9       9       0       0
P2      1       5       29      28      23      23
P3      2       2       12      10      8       8
P4      3       6       35      32      26      26
P5      5       3       17      12      9       9
P6      6       2       14      8       6       6
P7      7       4       24      17      13      13
P8      9       1       10      1       0       0
P9      11      3       20      9       6       6

Average WT = 10.11
Average TAT = 14.00
Number of switches = 8
```

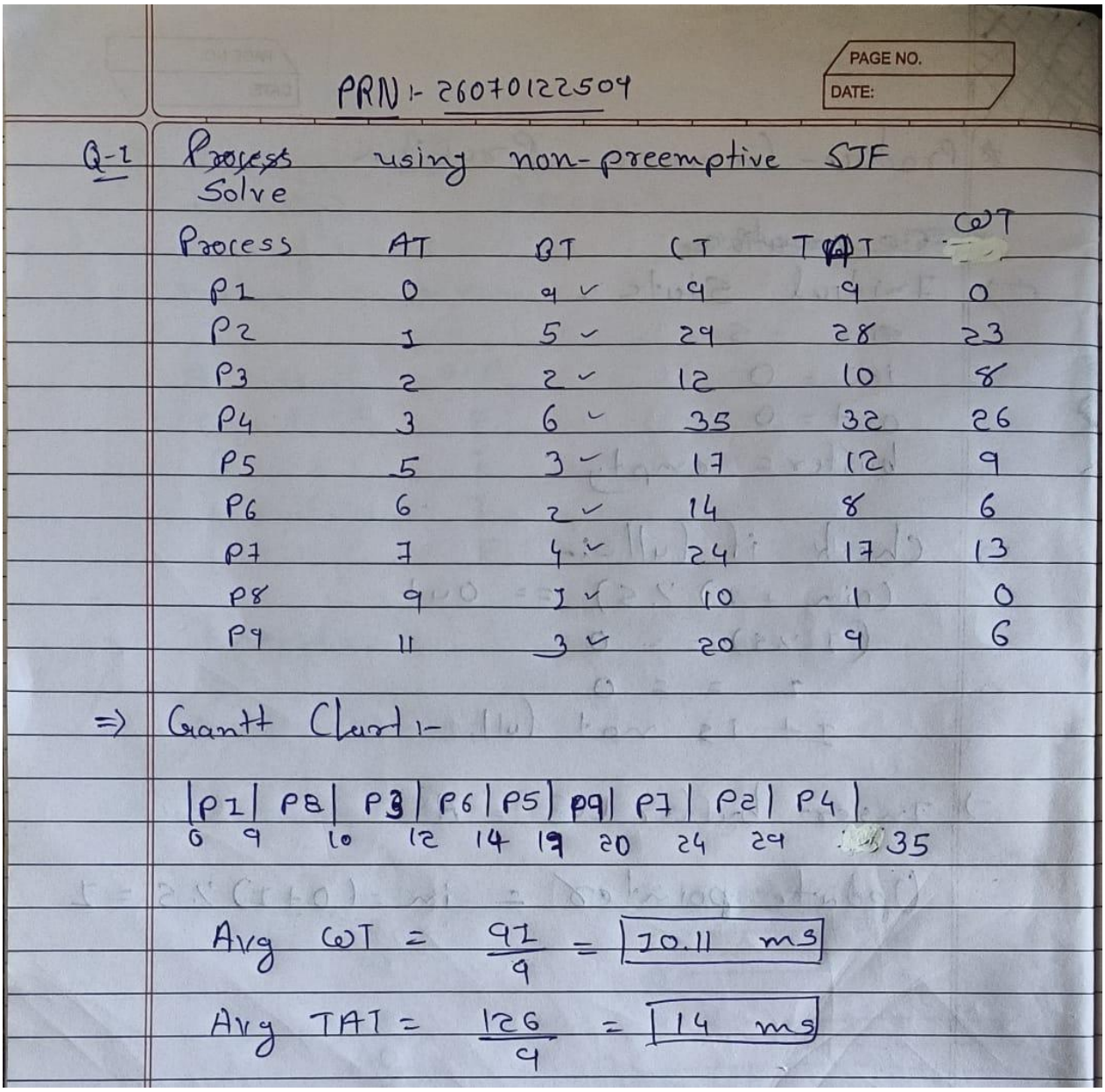
```
========== QUESTION 2 ==========

Gantt Chart:
| P1 | P3 | P5 | P7 | P4 | P2 | P6 |

Process AT      BT      CT      TAT     WT      RT
P1      0       7       7       7       0       0
P2      2       4       18      16      12      12
P3      4       1       8       4       3       3
P4      5       3       14      9       6       6
P5      6       2       10      4       2       2
P6      8       5       23      15      10      10
P7      10      1       11      1       0       0

Average WT = 4.71
Average TAT = 8.00
Number of switches = 6
```

Handwritten:



PRN :- 2607012250 4

PAGE NO.
DATE:

Q-1 Process / Solve using non-preemptive SJF

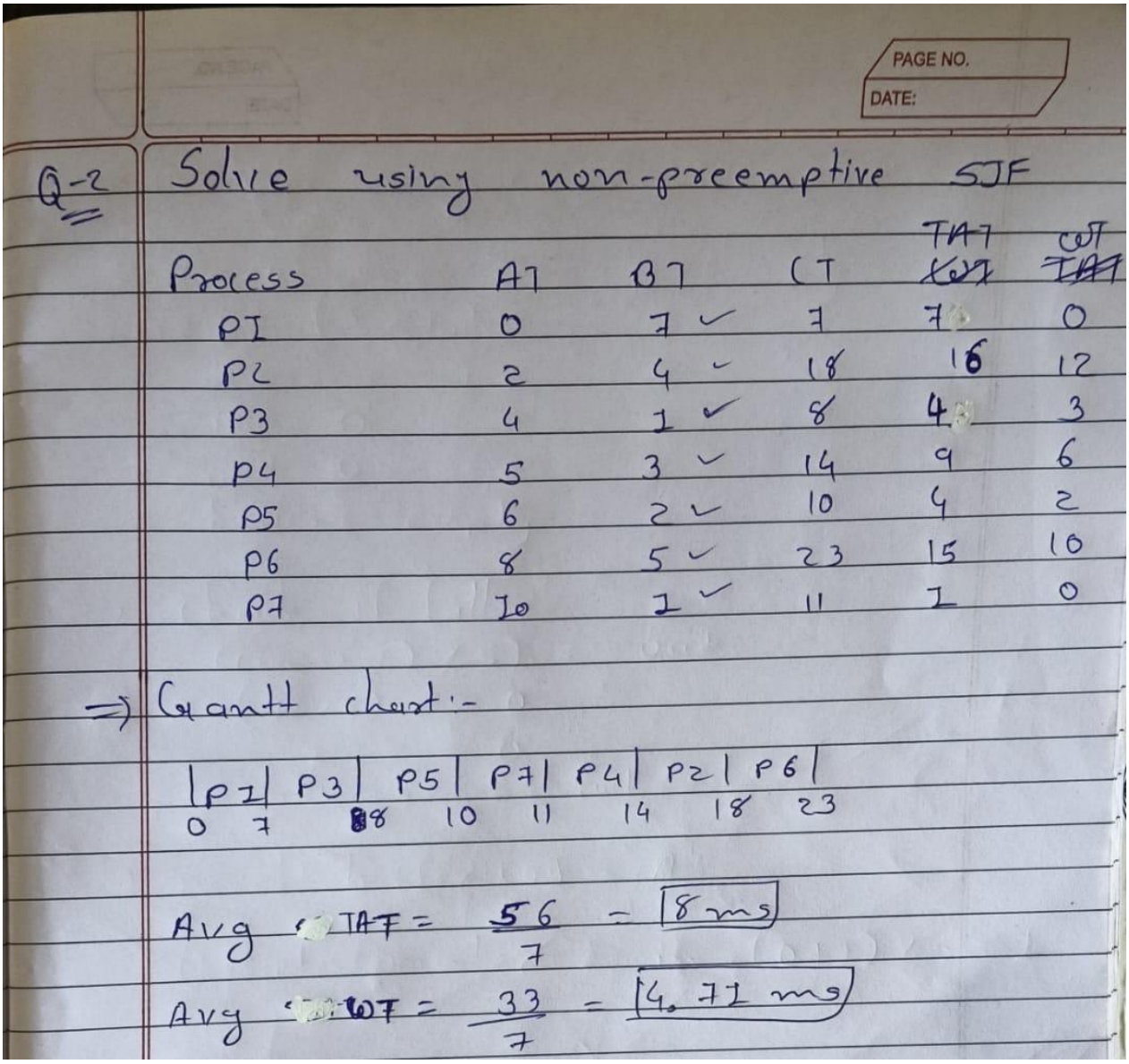
Process	AT	BT	CT	TAT	WT
P1	0	9	9	9	0
P2	1	5	29	28	23
P3	2	2	12	10	8
P4	3	6	35	32	26
P5	5	3	17	12	9
P6	6	2	14	8	6
P7	7	4	24	17	13
P8	9	1	10	1	0
P9	11	3	20	9	6

⇒ Gantt Chart :-

P1	P8	P3	P6	P5	P9	P7	P2	P4
0	9	10	12	14	17	20	24	29	35

Avg WT = $\frac{91}{9}$ = 10.11 ms

Avg TAT = $\frac{126}{9}$ = 14 ms



Q-2 Solve using non-preemptive SJF

Process	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	2	4	18	16	12
P3	4	1	8	4	3
P4	5	3	14	9	6
P5	6	2	10	4	2
P6	8	5	23	15	10
P7	10	1	11	1	0

⇒ Gantt chart:-

P1	P3	P5	P7	P4	P2	P6
0	7	8	10	11	14	18	23

Avg TAT = $\frac{56}{7}$ = 8 ms

Avg WT = $\frac{33}{7}$ = 4.71 ms

Conclusion:

We learned how to implement the Non-Preemptive SJF scheduling algorithm in C. We learned to calculate Completion Time, Turnaround Time, Waiting Time, and Response Time, and how to construct a Gantt Chart for process scheduling. We also calculated the Average Waiting Time and Average Turnaround Time. Overall, we understood how the SJF algorithm schedules processes based on their shortest burst time.

Assessment Schemes:

Attendance	Discipline	Short oral	Correctness of Lab Report	Timely completion of Lab Report	Total marks (10)	Signature of Teacher with Date